

Smart Commissioning App Detailed Specification

Generated: 11 June 2026

Repository: R:\Smart Commissioning App

Current local app: <http://127.0.0.1:8000>

API documentation: <http://127.0.0.1:8000/docs>

Current classification: working local review build / early MVP scaffold. Not production ready.

1. Executive Summary

The Smart Commissioning App, also referred to in the original specification as the Smart Commissioning Tool, is intended to support smart building commissioning and Master Systems Integrator workflows.

The app provides a structured web interface for configuring system connectivity, importing expected registers, discovering IP, BACnet, and MQTT data sources, validating live data, comparing BACnet source values with MQTT translated values, and producing commissioning evidence.

This repository currently contains a working React and FastAPI scaffold with important review-driven workflows implemented. It is suitable for demos, stakeholder review, workflow validation, API contract development, and continued engineering. It is not yet suitable for live production commissioning use as the authoritative system of record.

2. What The App Is For

The app is designed to replace disconnected manual commissioning activities across IP scanners, BACnet explorers, MQTT clients, spreadsheets, and ad hoc reports.

The app should allow commissioning teams to prove that:

- Expected devices are visible on the project network.
- Unexpected or rogue devices are identified.
- BACnet devices are discoverable.
- BACnet objects and point properties are available and meaningful.
- MQTT topics can be subscribed to.
- MQTT payloads are structured correctly.
- UDMI-style state, metadata, and pointset data is complete and current.

- Expected assets and points are present.
- Live values, units, timestamps, status flags, and quality indicators are valid.
- BACnet source values match MQTT translated values within defined tolerances.
- Commissioning evidence can be exported and forwarded.

Typical project contexts include:

- Building Management Systems.
- Energy metering.
- Lighting controls.
- Access control.
- CCTV and security interfaces.
- Fire monitoring interfaces where applicable.
- Lift and vertical transportation interfaces.
- IAQ sensors.
- IoT gateways.
- Occupancy and people-counting systems.
- MQTT brokers and smart building data platforms.

3. Target Users

MSI / Smart Commissioning Engineer

Primary user of the application.

Expected activities:

- Configure scan and protocol settings.
- Import project registers.
- Run discovery workflows.
- Review discovered devices and points.
- Inspect live data.
- Run validation checks.
- Raise or export issue registers.
- Produce commissioning evidence.

Software / Platform Engineer

Uses the output of the tool to troubleshoot protocol, payload, schema, and integration issues.

Expected activities:

- Review MQTT payloads.
- Review BACnet-to-MQTT mapping results.

- Investigate schema issues.
- Review missing, stale, or out-of-tolerance data.

Project Manager / Delivery Lead

Uses summary pages and reports to understand commissioning progress.

Expected activities:

- Review high-level readiness metrics.
- Track issue counts.
- Confirm evidence production status.
- Review handover reports.

Backend / System Administrator

Configures deployment, access, services, certificates, and storage.

Expected activities:

- Maintain broker credentials.
- Maintain certificates and keys.
- Configure backup and logging.
- Support network and protocol connectivity.

4. Current Application Status

Overall Status

The app is a local review build and early MVP scaffold. It has a working UI, API contracts, local runtime persistence, import templates, configuration validation, partial UDMI validation, and basic report artifact downloads.

It is not production ready because authentication, durable database storage, production worker execution, hardened secrets, protocol integrations, audit logs, monitoring, and formal evidence generation are not complete.

What Works Now

- React/Vite app served locally.
- FastAPI backend with OpenAPI docs.
- Application routes for Homepage, Configuration, IP Discovery, BACnet Discovery, MQTT Discovery, UDMI Payload Workbench, Data Validation, and Reports.
- Configuration snapshot retrieval, validation, and save.
- BACnet BBMD/Foreign Device lock rule.
- Server-side secret material endpoint returning masked references.
- Import profiles for project templates.
- Default CSV and XLSX template downloads.

- Import upload and row validation for CSV/XLSX files.
- Discovery and validation run contracts.
- Partial UDMI payload validation and MQTT config publish verification paths.
- Normalized validation issue shape.
- Basic DOCX, XLSX, and ZIP report download endpoints.
- Frontend build and backend unit tests pass.

What Is Still Mocked Or Partial

- Many visible table rows are demo/sample data.
- IP scanning is not yet a production scan engine.
- BACnet discovery is not yet connected to a real BACnet engine.
- MQTT discovery is not yet a full long-running broker discovery job.
- BACnet validation and mapping validation are API/job contracts but not complete production engines.
- Report files are basic generated artifacts, not complete formal commissioning evidence packs.
- Worker actors are mostly placeholders defining queue boundaries.

5. High-Level Workflow

The intended workflow is:

1. Open Homepage and review project readiness.
2. Configure network, BACnet, MQTT, certificate, time, backup, and logging settings.
3. Download default templates.
4. Fill expected project registers.
5. Import CSV or XLSX files.
6. Run IP Discovery.
7. Review reachable, missing, mismatched, and rogue network hosts.
8. Run BACnet Discovery.
9. Review BACnet devices, object counts, properties, reliability, and present values.
10. Run MQTT Discovery.
11. Review topics, payloads, timestamps, message counts, and extracted points.
12. Run Data Validation using the three main validation modes.
13. Use the UDMI Payload Workbench for deeper payload investigation when needed.
14. Review normalized issues and evidence outputs.
15. Generate Excel issue reports, Word handover reports, and evidence packs.

6. Application Modules

Homepage

Purpose:

- Provides the starting workspace and project overview.
- Shows project readiness, asset counts, online assets, open issues, evidence packs, workflow stages, active assets, issue summary, and run status.

Current status:

- Implemented as a React dashboard using demo/sample data.
- Useful for review and product direction.
- Needs connection to persisted project, run, issue, and evidence data.

Configuration

Purpose:

- Stores network, BACnet, MQTT, certificate, time, backup, and logging settings required before discovery and validation.

Current implemented behavior:

- Loads configuration from the backend API.
- Saves validated configuration.
- Validates IP addresses, subnet mask, DNS servers, ports, BACnet network number, TTL, MQTT broker fields, keep-alive, certificate references, backup fields, and enabled/disabled fields.
- Supports CA certificate, client certificate, and private key paste or file selection.
- Stores supported secret material server-side and returns masked secret:// references.
- BBMD enabled locks Foreign Device to Disabled in the UI.
- Backend rejects snapshots where BBMD and Foreign Device are both enabled.

Important production requirement:

- Do not store secrets in browser storage.
- Move from local file-backed storage to secure storage/vault references.
- Add audit logs for all configuration changes.

IP Discovery

Purpose:

- Imports the expected network device register.
- Scans configured ports and protocols.
- Identifies reachable, missing, mismatched, and rogue hosts.

Current implemented behavior:

- Import profile exists for ip_register.
- Default CSV and XLSX templates are downloadable.
- UI exposes port/protocol selection.
- Common fallback ports include 47808/udp, 80/tcp, and 443/tcp.

- Demo result table shows asset, expected IP, observed state, MAC address, ports, match basis, last seen, detailed status, and result.
- IP page now emphasizes the default import template rather than network-level issues/evidence.

Production gaps:

- Integrate real scan engine.
- Persist scan runs and raw evidence.
- Store normalized observations and register comparison results.
- Provide safe network authorization controls before active scans.

BACnet Discovery

Purpose:

- Discovers BACnet devices and object lists.
- Reads object/property detail.
- Compares discovered devices and points against expected registers.

Current implemented behavior:

- Import profiles exist for bacnet_register and bacnet_points.
- Discovery run endpoint contract exists.
- UI shows BACnet devices, instances, object counts, last discovered time, detailed status, and result.
- Detail inspector explains the target drilldown: object type, instance, object name, present value, units, reliability, status flags, priority array, and timestamp.

Production gaps:

- Integrate a BACnet engine such as BAC0 or bacpypes3.
- Persist device and object records.
- Add object-level drilldown with pagination/search.
- Capture raw evidence and property-read samples.

MQTT Discovery

Purpose:

- Connects to a configured MQTT broker.
- Subscribes to expected or discovered topics.
- Captures payloads and extracts points.
- Compares observed topics and payloads against the MQTT register.

Current implemented behavior:

- Import profiles exist for mqtt_register and mqtt_points.
- Discovery run endpoint contract exists.
- UI shows topic, asset, payload last seen, message count, detailed connection status, copy payload action, and result.

- Detail inspector explains live data expectations: decoded JSON, extracted point names, present values, units, timestamp freshness, and schema warnings.

Production gaps:

- Implement long-running broker subscription service.
- Store payload samples and extracted points.
- Implement topic discovery, payload schema classification, and stale/silent topic detection.
- Harden TLS/certificate handling.

UDMI Payload Workbench

Purpose:

- Provides a technical workbench for state, metadata, pointset, and controlled MQTT config publish validation.
- Supports deeper investigation than the normal operator validation page.

Current implemented behavior:

- Accepts expected schedule JSON.
- Accepts state payload JSON.
- Accepts metadata payload JSON.
- Accepts pointset payload JSON.
- Can pass live broker topic/capture parameters into validation.
- Provides a controlled publish form with confirmation gate.
- Can verify next pointset payload against expected point/value.
- Emits normalized issues for payload/schema/state/metadata/pointset problems.

Production gaps:

- Continue porting logic from device_udmi_payload_validation/.
- Remove remaining fixture/local fallback assumptions.
- Persist per-device evidence artifacts.
- Handle gateway-style state-only devices.
- Add stronger UX around raw JSON vs operator summaries.

Data Validation

Purpose:

- Main validation workspace.
- Groups validation into three human-readable modes.

Current validation modes:

- MQTT Payload Check: checks MQTT/UDMI payload structure, timestamps, state, metadata, reporting cadence, and live values.
- BACnet Point Check: checks BACnet point names, object details, reliability, units, and present values.

- BACnet vs MQTT Comparison: compares matching live BACnet and MQTT values using mapping and tolerance templates.

Current implemented behavior:

- Import profiles exist for asset_validation, bacnet_points, mqtt_points, mapping, and tolerances.
- UI exposes all three validation actions.
- Demo result table shows asset, point, BACnet value, MQTT value, tolerance, and result.
- Detail inspector explains mapping logic, unit conversion, tolerance, pass/warn/fail, and latest timestamps.

Production gaps:

- Implement BACnet point validation engine.
- Implement mapping/tolerance comparison engine.
- Persist validation results and point-level evidence.
- Clearly separate not-tested, not-applicable, warning, and fail states.

Reports

Purpose:

- Generates issue reports, evidence packs, and handover outputs.

Current implemented behavior:

- Report API accepts output_format values zip, xlsx, and docx.
- UI exposes Excel Report and Word Report actions.
- Report download endpoint returns basic XLSX, DOCX, or ZIP artifacts.
- Demo report queue shows Excel issue report, Word handover report, evidence pack, and blocked report examples.

Production gaps:

- Generate full formal report content from stored run records.
- Include raw evidence references.
- Include validation summaries, issue lists, screenshots/logs where useful, and project metadata.
- Add filtered exports tied to run IDs and selected systems.

7. API Surface

Health and blueprint:

- GET /api/v1/health
- GET /api/v1/blueprint

Configuration:

- GET /api/v1/configuration
- PUT /api/v1/configuration

- POST /api/v1/configuration/validate
- POST /api/v1/configuration/secrets

Imports:

- GET /api/v1/imports/profiles
- GET /api/v1/imports/templates/{import_type}.{csv|xlsx}
- POST /api/v1/imports
- GET /api/v1/imports/{import_id}
- GET /api/v1/imports/{import_id}/errors

Discovery:

- POST /api/v1/discovery/ip/runs
- POST /api/v1/discovery/bacnet/runs
- POST /api/v1/discovery/mqtt/runs
- GET /api/v1/discovery/runs
- GET /api/v1/discovery/runs/{run_id}
- GET /api/v1/discovery/runs/{run_id}/results

Validation:

- POST /api/v1/validation/udmi/runs
- POST /api/v1/validation/mqtt-config/runs
- POST /api/v1/validation/bacnet/runs
- POST /api/v1/validation/mapping/runs
- GET /api/v1/validation/runs
- GET /api/v1/validation/runs/{run_id}
- GET /api/v1/validation/runs/{run_id}/issues

Reports:

- POST /api/v1/reports
- GET /api/v1/reports
- GET /api/v1/reports/{report_id}
- GET /api/v1/reports/{report_id}/download

8. Import Templates

Default templates are generated by the backend from the import profiles. This means the downloadable format stays aligned with the validator.

Supported import types:

- ip_register
- bacnet_register

- mqtt_register
- asset_validation
- bacnet_points
- mqtt_points
- mapping
- tolerances

Template columns:

- ip_register: Project/site; System; Asset ID; Asset name; Expected IP address; Expected hostname; Expected services/ports.
- bacnet_register: Project/site; System; Asset ID; Asset name; BACnet device instance; BACnet network; IP address.
- mqtt_register: Project/site; System; Asset ID; Asset name; Expected topic; Payload type; Expected schema version; Expected points; Expected units; Expected reporting interval; Source protocol; Notes.
- asset_validation: Project/site; System; Asset ID; Asset name; Source protocol; Expected online status; Expected topic or device reference; Location.
- bacnet_points: Asset ID; Device instance; BACnet network; Object type; Object instance; Object name; Expected point name; Expected units; Expected value type; Required/optional flag.
- mqtt_points: Asset ID; Topic; Payload type; JSON path or field name; Expected point name; Expected units; Expected value type; Required/optional flag; Expected reporting interval.
- mapping: Asset ID; BACnet device instance; BACnet object type; BACnet object instance; BACnet object name; BACnet units; MQTT topic; MQTT field/path; MQTT units; Tolerance; Mapping required flag.
- tolerances: Asset ID; Point name; Tolerance.

9. Architecture

Current scaffold:

- frontend/: React, TypeScript, Vite, React Router, TanStack Query.
- backend/: FastAPI API, Pydantic schemas, runtime services.
- worker/: background job actors and queue boundary.
- infra/: Docker Compose direction for API, worker, Postgres, Redis, and object storage.
- device_udmi_payload_validation/: standalone source utility for UDMI/MQTT payload validation behavior.
- backend/runtime/: local file-based bootstrap storage.

Target production architecture:

- React UI.
- FastAPI API.
- PostgreSQL for persistent domain data.
- Redis for queue/cache/transient state.
- Dramatiq workers for discovery, validation, and reports.
- Object storage for raw imports, rejected-row reports, payload evidence, and report artifacts.
- Server-Sent Events or WebSockets for live job progress.

10. Shared Domain Model

The backend should converge on these entities:

- project
- site
- system
- device
- asset
- import_batch
- imported_file
- import_error
- ip_scan_run
- ip_scan_result
- bacnet_discovery_run
- bacnet_device
- bacnet_object
- bacnet_property_sample
- mqtt_discovery_run
- mqtt_topic
- mqtt_payload
- mqtt_extracted_point
- validation_run
- validation_result
- mapping_result
- issue
- report
- report_file
- evidence_artifact
- user_note
- configuration_snapshot
- secret_reference
- audit_log

11. Normalized Issue Model

All discovery and validation paths should emit one shared issue contract.

Recommended fields:

- issue_id
- run_id

- project_id
- site_id
- asset_id
- device_id
- source
- issue_type
- severity
- status
- point_name
- topic
- bacnet_object_ref
- description
- expected_value
- observed_value
- tolerance
- suggested_action
- raw_evidence_uri
- created_at
- resolved_at

Why this matters:

- UDMI, MQTT, BACnet, IP, mapping, and reporting should not each invent separate issue formats.
- Reports need one consistent issue register.
- Operators need one place to triage commissioning defects.

12. Security And Secret Handling

Current state:

- Secret material can be submitted to the backend.
- Backend stores supported certificate/key material server-side.
- API returns masked secret references.
- MQTT password is masked in UI.

Production requirements:

- Use a production-grade vault or encrypted secret store.
- Never store private keys or passwords in browser storage.
- Audit configuration and secret changes.
- Restrict secret access by role.
- Harden TLS configuration for MQTT.
- Add authentication and authorization before live deployment.

13. Background Jobs

Current job names:

- ip_discovery
- bacnet_discovery
- mqtt_discovery
- udmi_validation
- mqtt_config_publish
- bacnet_validation
- mapping_validation
- report_generation

Expected statuses:

- queued
- running
- succeeded
- failed
- cancelled

Expected job data:

- run_id
- job_type
- project_id
- site_id
- parameters
- status
- stage
- progress_percent
- result_summary
- issues
- error_message
- timestamps

Production requirement:

- Long-running jobs must not block the UI.
- Jobs should publish progress.
- Jobs should store raw evidence and normalized results.
- Jobs should be retryable where safe.

14. Reports And Evidence

Report types currently represented:

- IP discovery.
- BACnet discovery.
- MQTT discovery.
- UDMI validation.
- Data validation.
- Issue report.
- Evidence pack.

Output formats currently supported by the API:

- xlsx
- docx
- zip

Production report expectations:

- Excel report for filtering, issue triage, and technical review.
- Word report for formal commissioning handover.
- Evidence pack for raw artifacts, payload samples, logs, and machine-readable results.
- Reports must be generated from stored run records, not live screen state.

15. Production Readiness Assessment

Current verdict: not production ready.

The app is appropriate for:

- Local review.
- Stakeholder walkthroughs.
- UI workflow validation.
- API contract review.
- Continued feature development.
- Controlled technical demos.

The app is not yet appropriate for:

- Live production commissioning as the source of truth.
- Unsupervised site deployment.
- Handling production secrets without further hardening.
- Formal evidence handover without report-content completion.
- Multi-user controlled operation.

16. Current Limitations

- No authentication or role-based access control.
- No user/session model.
- No production tenancy model.
- Local file-based runtime storage instead of PostgreSQL.
- No production object storage integration.
- No complete audit log.
- No production backup/restore implementation.
- No full real IP scan engine integration.
- No full real BACnet discovery engine integration.
- No full MQTT discovery worker implementation.
- No complete BACnet point validation engine.
- No complete BACnet-to-MQTT mapping validation engine.
- Reports are basic generated artifacts, not final evidence packs.
- Many UI result rows still use demo/sample data.
- Worker actors are mostly placeholders.
- Monitoring, alerting, logging, and deployment runbooks are incomplete.
- CI/CD is not complete.
- Browser UI tests and E2E tests are not complete.
- Kluster.ai verification could not run in the latest session because the tool reported a connection/sign-in error.

17. Production Roadmap

Phase 1 - Product Hardening

Outcome:

- Stable MVP suitable for controlled pilot use.

Main work:

- Persist configuration, imports, runs, issues, and reports in PostgreSQL.
- Finish normalized issue model.
- Add migrations.
- Stabilize job progress and result APIs.
- Add integration and E2E tests.

Phase 2 - Protocol Integration

Outcome:

- Real discovery and validation against site networks.

Main work:

- Integrate nmap or equivalent IP scanning.
- Integrate BACnet engine.
- Integrate MQTT discovery service.
- Complete UDMI live capture.
- Complete BACnet point validation.
- Complete BACnet-to-MQTT mapping and tolerance engine.

Phase 3 - Evidence And Reports

Outcome:

- Forwardable commissioning evidence packs.

Main work:

- Build formal DOCX report content.
- Build technical XLSX issue registers.
- Build evidence ZIP with raw artifacts.
- Include source run IDs, project metadata, issue summaries, and raw evidence references.

Phase 4 - Security And Operations

Outcome:

- Deployment-ready controlled system.

Main work:

- Add authentication.
- Add RBAC.
- Add audit logging.
- Add vault/key management.
- Harden TLS and certificate handling.
- Add monitoring, backups, structured logging, and retention policies.

Phase 5 - Production Deployment

Outcome:

- Repeatable installation and support model.

Main work:

- Docker Compose pilot packaging.
- Database migrations.
- Deployment runbooks.
- CI/CD.

- Acceptance test scripts.
- Security review.

18. AI Agent Handoff Notes

An AI agent continuing this project should read these files first:

- README.md: repository overview and production direction.
- Smart Commissioning Tool Specification.pdf: original product specification.
- spec_extracted.txt: extracted text version of the original specification.
- docs/production-architecture.md: architecture direction and production gaps.
- docs/v1-review-checklist.md: prior review status.
- frontend/src/app/App.tsx: navigation and page shell.
- frontend/src/features/workflow/moduleData.ts: module definitions, imports, run actions, and next steps.
- frontend/src/features/workflow/ModulePage.tsx: shared UI for imports, templates, runs, results, inspectors, UDMI workbench, and reports.
- frontend/src/features/workflow/ConfigurationPage.tsx: configuration UI and BBMD/Foreign Device lock.
- frontend/src/features/workflow/operatorData.ts: current demo/sample UI data.
- frontend/src/api/client.ts: frontend API client contracts.
- backend/app/api/routes: FastAPI route implementations.
- backend/app/services: configuration, import, run, UDMI, MQTT publish, and report services.
- backend/app/schemas: Pydantic API contracts.
- backend/tests/test_v1_review_contracts.py: current review-contract tests.
- worker/app/tasks.py: worker queue boundary.
- infra/docker-compose.yml: intended local service stack.

Development guidance for an AI agent:

- Treat frontend/, backend/, worker/, and infra/ as the production scaffold.
- Treat static prototypes as reference material only.
- Keep UI changes aligned with the existing operational dashboard style.
- Keep validation and reporting traceable to imports, run IDs, source data, and evidence artifacts.
- Use structured APIs and schemas instead of ad hoc string contracts.
- Add tests when changing contracts or backend behavior.
- For complex multi-file work, follow the repo's Ruflo guidance.
- If Kluster tools are available, run the required Kluster verification after file changes.

Useful commands:

- Start backend app: `cd backend && python -m uvicorn app.main:app --host 127.0.0.1 --port 8000`
- Start frontend dev server: `cd frontend && npm run dev`
- Build frontend: `cd frontend && npm run build`
- Run backend tests: `python -m unittest backend.tests.test_v1_review_contracts`

- Open API docs: <http://127.0.0.1:8000/docs>

19. Human Review Guide

For a non-developer stakeholder:

- Start at Homepage to understand readiness and workflow.
- Open Configuration to review required connectivity settings.
- Open IP Discovery to see how default templates and network scan results will work.
- Open BACnet Discovery to understand device and object-level drilldown intent.
- Open MQTT Discovery to inspect topics, payloads, message counts, and live data intent.
- Open Data Validation to see the three clear validation modes.
- Open UDMI Payload Workbench only for deeper technical payload testing.
- Open Reports to see Excel and Word report directions.

For a technical reviewer:

- Review /docs for API contracts.
- Review import templates.
- Review backend services and schemas.
- Review test coverage.
- Review current limitations before assuming production readiness.

20. Forwarding Summary

This PDF is a current-state specification and implementation handoff. It explains what the Smart Commissioning App is intended to become, what exists now, what is incomplete, and how a human or AI engineer should continue the work.

The most important message to forward is: this is a strong local review/MVP scaffold with useful UI and API foundations, but it is not production ready until real protocol engines, durable persistence, security, worker execution, audit logging, and formal evidence generation are completed.